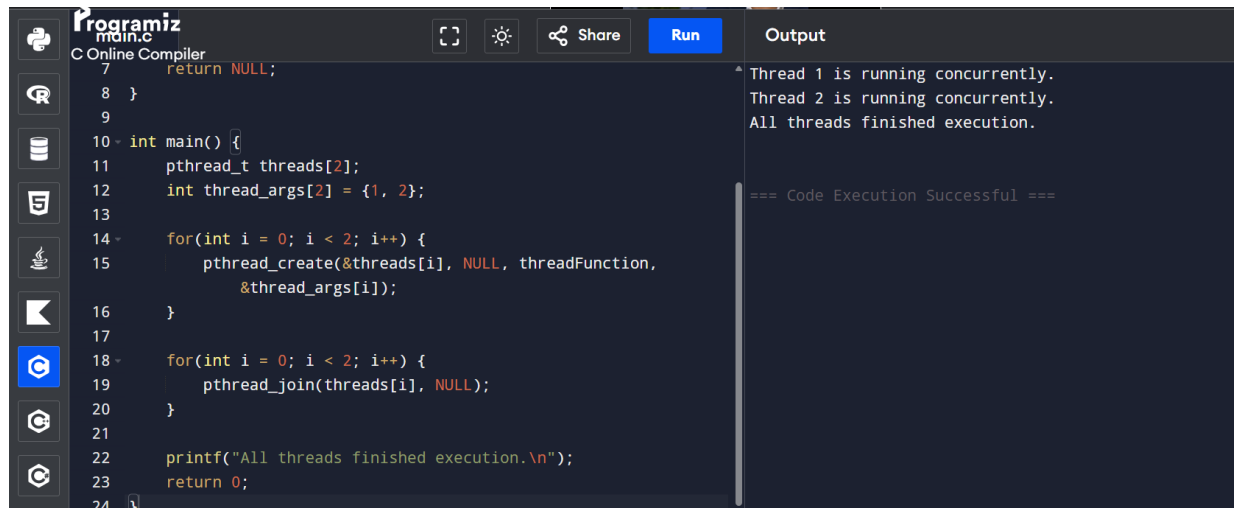


# EXPERIMENT 11:

11. Illustrate the concept of multithreading using a C program.



The screenshot shows the Programiz C Online Compiler interface. The code editor on the left contains a C program that creates two threads and joins them. The output panel on the right shows the execution results.

```
7     return NULL;
8 }
9
10 int main() {
11     pthread_t threads[2];
12     int thread_args[2] = {1, 2};
13
14     for(int i = 0; i < 2; i++) {
15         pthread_create(&threads[i], NULL, threadFunction,
16                       &thread_args[i]);
17     }
18
19     for(int i = 0; i < 2; i++) {
20         pthread_join(threads[i], NULL);
21     }
22
23     printf("All threads finished execution.\n");
24     return 0;
25 }
```

Output:

```
Thread 1 is running concurrently.
Thread 2 is running concurrently.
All threads finished execution.

=== Code Execution Successful ===
```

## PSEUDO CODE :

1. Define thread handler function.
2. In main, declare pthread\_t variables.
3. Call pthread\_create() to start threads executing handler function.
4. Call pthread\_join() to suspend main until threads complete.

## CODE:

```
#include <stdio.h>

#include <pthread.h>

void* threadFunction(void* arg) {

    int id = *(int*)arg;

    printf("Thread %d is running concurrently.\n", id);

    return NULL;

}

int main() {

    pthread_t threads[2];

    int thread_args[2] = {1, 2};

    for(int i = 0; i < 2; i++) {

        pthread_create(&threads[i], NULL, threadFunction, &thread_args[i]);

    }

    for(int i = 0; i < 2; i++) {

        pthread_join(threads[i], NULL);

    }

    printf("All threads finished execution.\n");

    return 0;

}
```